Practical - 01 : Breadth First Search (BFS) & Iterative Depth First Search (DFS)

Aim:

- To implement BFS for problem solving.
- To implement IDFS for the same problem.
- To compare the performance and efficiency of BFS and IDFS.

1A) Breadth First Search Algorithm(BFS)

Code:

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        vertex = queue.popleft()
        if vertex not in visited:
            visited.add(vertex)
            print(vertex)
            # Explore neighbors
            neighbors = graph[vertex]
            for neighbor in neighbors:
                if neighbor not in visited:
                    queue.append(neighbor)
# Example usage
graph = {
    "A": ["B", "C"],
    "B": ["A", "D", "E"],
    "C": ["A", "F"],
    "D": ["B"],
    "E": ["B", "F"],
    "F": ["C", "E"],
}
start_vertex = "A"
bfs(graph, start_vertex)
```

Output:



1B) Iterative Deapth First Search

Code:

```
# Python program to perform Iterative
# Depth First Traversal of Graph
def dfs(adj):
    n = len(adj)
    visited = [False] * n
    res = []
    # Start DFS from node 0.
    st = []
    st.append(0)
    while st:
        node = st.pop()
        # If node is already visited, continue
        if visited[node] == True:
            continue
        # Mark this node as visited
        visited[node] = True
        res.append(node)
        # Traverse all edges (as stack is used, so
        size = len(adj[node])
        for i in range(size - 1, -1, -1):
            v = adj[node][i]
            if not visited[v]:
                st.append(v)
    return res
if __name__ == "__main__":
    adj = [[1, 2], [2], [0, 3], [3]]
    res = dfs(adj)
    for node in res:
        print(node, end=" ")
    print()
```

Output:

```
•••   0 1 2 3
```

Practical No - 02 : A* Search Algorithm and Recursive Best-First Search

Aim:

- To implement the A* Search algorithm.
- To implement the Recursive Best-First Search algorithm.
- To compare both algorithms based on performance and effectiveness.

2A) A* Search Algorithm

Code:

```
def astaralgo(start_node,stop_node):

   open_set = set(start_node)
   closed_set= set()

   g={}
   parents = {}
   g[start_node] = 0
   parents[start_node]  = start_node

   while len(open_set) > 0:
      n=None

      for v in open_set:
         if n==None or g[v] + heuristic(v) < g[n] + heuristic(n):
            n=v
      if n==stop_node or Graph_nodes[n]==None:
            pass

      else:

         for(m,weight) in get_neighbors(n):
            if m not in open_set  and m not in closed_set:
                  open_set.add(m)
                  parents[m]=n
                  g[m] = g[n] + weight
            else:
                  if g[m] > g[n] +  weight:
                     g[m] = g[n] + weight
                     parents[m]=n

                     if m in closed_set:
                        closed_set.remove(m)
                        open_set.add(m)
      if n==None:
         print("path does not exist!")
         return None

      if n==stop_node:
```

```
            path=[]

            while parents[n]!=n:
                path.append(n)
                n=parents[n]

            path.append(start_node)

            path.reverse()

            print("Path found: {}".format(path))
            return path


        open_set.remove(n)
        closed_set.add(n)
    print("Path does not exist!")
    return None


def get_neighbors(v):
    if v in Graph_nodes:
        return Graph_nodes[v]

    else:
        return None

def heuristic(n):
    H_dist = {
        'A' : 11,
        'B' : 6,
        'C' : 99,
        'D' : 1,
        'E' : 7,
        'G' : 0,
    }
    return H_dist[n]


Graph_nodes = {
    'A' : [('B',2),('E',3)],
    'B' : [('C',1),('G',9)],
    'C' : None,
    'E' : [('D',6)],
    'D' : [('G',1)],
}
astaralgo('A','G')
```

Output:

```
Path found: ['A', 'E', 'D', 'G']
['A', 'E', 'D', 'G']
```

2B)Recursive Best-First Search

Code:

```
from queue import PriorityQueue

class Node:

 def __init__(self, state, parent=None, f=float('inf')):

      self.state = state

      self.parent = parent

      self.f = f

def rbfs(start, goal):

  f_limit = float('inf')

  stack = [(Node(start, f=0), f_limit)]

  visited = set()

  while stack:

     (node, f) = stack.pop()

     visited.add(node.state)

     if node.state == goal:

        path = []

        cost = node.f

        while node is not None:

           path.append(node.state)

           node = node.parent

        return list(reversed(path)), cost

     successors = []

     for neighbor, cost in get_neighbors(node.state):

        if neighbor not in visited:

           child = Node(neighbor, parent=node)

           child.f = max(child.parent.f, cost)

           successors.append(child)

     if len(successors) == 0:

        continue

     successors.sort(key=lambda x: x.f)
```

```
        best = successors[0]
        if best.f > f_limit:
            return None, best.f
        alternative = successors[1].f if len(successors) > 1 else float('inf')
        stack.append((best, min(f_limit, alternative)))
    return None, float('inf')
def get_neighbors(state):
    successors = {
        1: [(2, 3), (3, 5)],
        2: [(1, 3), (4, 7)],
        3: [(1, 5), (5, 2)],
        4: [(2, 7), (6, 4)],
        5: [(3, 2), (7, 6)],
        6: [(4, 4), (8, 8)],
        7: [(5, 6), (8, 5)],
        8: [(6, 8), (7, 5)],
    }   return successors.get(state, [])
if __name__ == '__main__':
    start_state = 1
    goal_state = 8
    path, cost = rbfs(start_state, goal_state)
    if path is not None:
        print(f"Optimal path from {start_state} to {goal_state}:")
        print(" -> ".join(map(str, path)))
        print(f"Total cost: {cost}")
    else:
        print("No path found.")
```

Output:

```
Optimal path from 1 to 8:
1 -> 2 -> 4 -> 6 -> 8
Total cost: 8
```

Practical No - 03 : Decision Tree Learning

Aim:

- To construct a decision tree using a given dataset.
- To test the decision tree on test data.
- To evaluate its accuracy and effectiveness.
- To visualize and interpret the generated decision tree.

Code:

```
import numpy as np

import matplotlib.pyplot as plt

import pandas as pd

from sklearn.metrics import accuracy_score

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree


# Load dataset

data = pd.read_csv('Iris.csv')


# Drop 'Target' to create feature set X, and 'ID' so it isn't treated as a feature

X = data.drop(columns=['Target', 'ID'], errors='ignore')

y = data['Target']


# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(

  X, y, test_size=0.2, random_state=42

)


# Initialize and fit the Decision Tree classifier

clf = DecisionTreeClassifier(random_state=42)

clf.fit(X_train, y_train)
```

```
# Evaluate accuracy

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.2f}")


# Plot Decision Tree

plt.figure(figsize=(12, 8))

plot_tree(

  clf,

  filled=True,

  feature_names=list(X.columns),

  class_names=[str(c) for c in y.unique()],

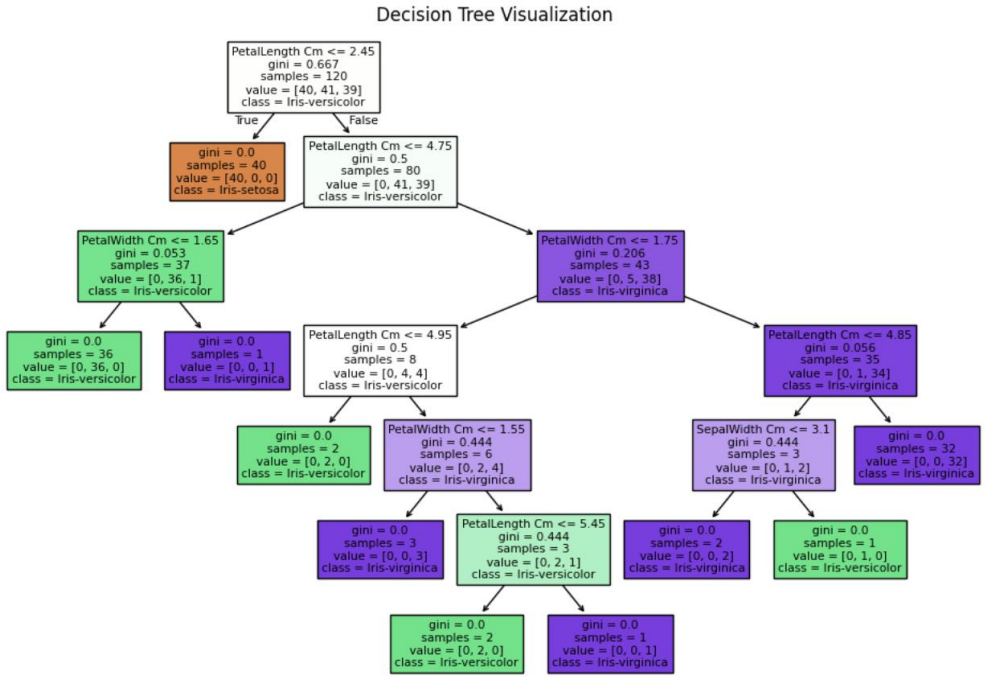
)

plt.title("Decision Tree Visualization")

plt.show()
```

Output:

```
Accuracy: 1.00
```



Practical No - 04 : Support Vector Machine(SVM)

Aim:

- To implement the SVM algorithm.
- To train an SVM model using a given dataset.
- To optimize the parameters of the SVM model.
- To evaluate the model using test data.
- To analyze the obtained classification results.

Code:

```
# Import necessary libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.svm import SVC

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

# Load your dataset from a local file (e.g., CSV)

# Replace 'your_dataset.csv' with the actual path to your dataset file

data = pd.read_csv('Iris.csv')

# Assuming the target variable is in a column named 'target'

X = data.drop('Target', axis=1)

y = data['Target']

# Split the dataset into a training set and a testing set

X_train, X_test, y_train, y_test = train_test_split(X, y,

test_size=0.2, random_state=42)

# Create an SVM classifier

svm_classifier = SVC(kernel='linear') # You can choose different kernels here

# Fit the classifier to the training data

svm_classifier.fit(X_train, y_train)

# Make predictions on the test data

y_pred = svm_classifier.predict(X_test)
```

```
# Calculate the accuracy of the model

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.2f}")
```

Output:

```
...  Accuracy: 1.00
```

Practical No - 5 : Adaboost Ensemble Learning

Aim:

- To create an ensemble using weak classifiers.
- To train the ensemble model on a given dataset.
- To evaluate the performance of the ensemble model.
- To compare the ensemble results with individual weak classifiers.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.svm import SVC
from sklearn import metrics
iris = load_iris()
X = iris.data
y = iris.target


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
svc = SVC(probability=True, kernel='linear')

abc = AdaBoostClassifier(n_estimators=50, estimator=svc, learning_rate=1.0, random_state=42)
model = abc.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

Output:

```
...  Accuracy: 0.9777777777777777
```

Practical No - 6 : Naive Bayes' Classifier

Aim:

- To implement the Naive Bayes algorithm.
- To train a Naive Bayes model using a given dataset.
- To calculate the class probabilities.
- To evaluate the accuracy of the model on test data.
- To analyze the classification results.

Code:

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score

# Load the Iris dataset

iris = load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

random_state=42)

clf = GaussianNB()

# Train the classifier on the training data

clf.fit(X_train, y_train)

# Make predictions on the test data

y_pred = clf.predict(X_test)

# Calculate and print the accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Output:

```
... Accuracy: 1.0
```

Practical No - 7 : K-Nearest Neighbors (K-NN)

Aim:

- To implement the K-NN algorithm.
- To apply K-NN to a given dataset.
- To predict the class or value for test data.
- To evaluate the accuracy or error of the predictions.
- To analyze the obtained results.

Code:

```
# Import necessary modules

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split

from sklearn.datasets import load_iris

import numpy as np

import matplotlib.pyplot as plt

irisData = load_iris()

# Create feature and target arrays

X = irisData.data

y = irisData.target

# Split into training and test set

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size = 0.2, random_state=42)

neighbors = np.arange(1, 9)

train_accuracy = np.empty(len(neighbors))

test_accuracy = np.empty(len(neighbors))

# Loop over K values

for i, k in enumerate(neighbors):

 knn = KNeighborsClassifier(n_neighbors=k)

 knn.fit(X_train, y_train)

  # Compute training and test data accuracy
```

```
 train_accuracy[i] = knn.score(X_train, y_train)

 test_accuracy[i] = knn.score(X_test, y_test)


# Generate plot

plt.plot(neighbors, test_accuracy, label = 'Testing dataset Accuracy')

plt.plot(neighbors, train_accuracy, label = 'Training dataset Accuracy')


plt.legend()

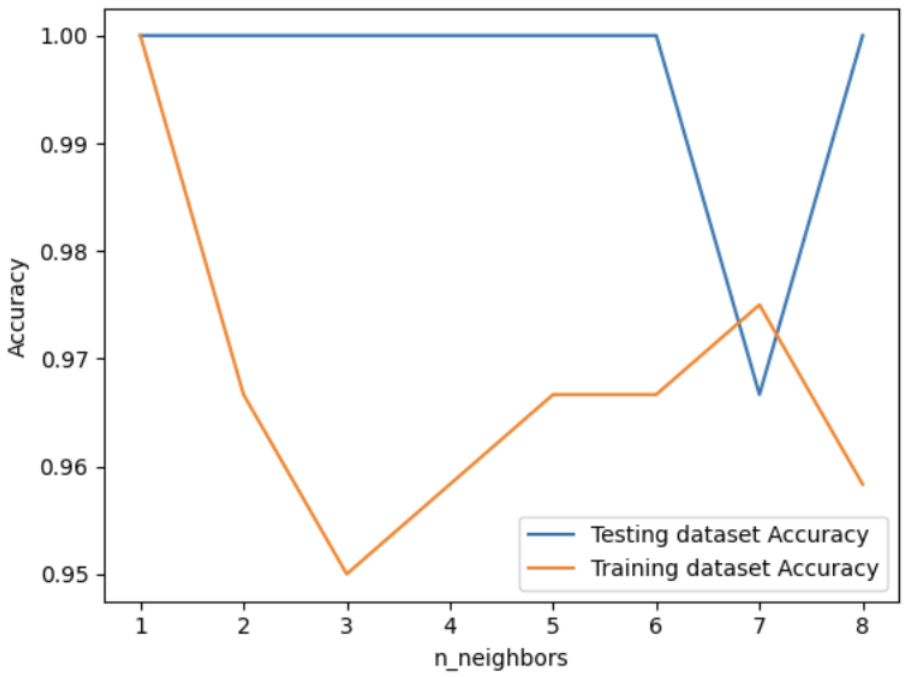
plt.xlabel('n_neighbors')

plt.ylabel('Accuracy')

plt.show()
```

Output:

```
...
```



Practical No - 8 : Association Rule Mining

Aim:

- To find frequent itemsets using the Apriori algorithm.
- To generate association rules from frequent itemsets.
- To calculate the support and confidence of the rules.
- To interpret and analyze the discovered association rules.

Code:

```
# 1. Install the required library (only needs to be run once per session)

!pip install mlxtend

from mlxtend.frequent_patterns import apriori, association_rules

from mlxtend.preprocessing import TransactionEncoder

import pandas as pd

# 2. Prepare data

transactions = [['bread', 'milk', 'eggs'],

        ['bread', 'butter'],

        ['milk', 'butter', 'cheese'],

        ['bread', 'milk', 'butter']]

te = TransactionEncoder()

te_array = te.fit(transactions).transform(transactions)

df = pd.DataFrame(te_array, columns=te.columns_)

print("--- One-Hot Encoded DataFrame ---")

print(df)

print("\n" + "="*50 + "\n")

# 3. Generate frequent itemsets

frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)

print("--- Frequent Itemsets ---")

print(frequent_itemsets)

print("\n" + "="*50 + "\n")

# 4. Generate rules
```

```
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)


print("--- Association Rules ---")

# Using display() instead of print() in Colab makes the table look much cleaner!

display(rules)
```

Output:

```
--- One-Hot Encoded DataFrame ---
   bread  butter  cheese   eggs   milk
0   True   False   False   True   True
1   True    True   False  False  False
2  False    True    True  False   True
3   True    True   False  False   True

==================================================

--- Frequent Itemsets ---
   support         itemsets
0     0.75          (bread)
1     0.75         (butter)
2     0.75           (milk)
3     0.50  (bread, butter)
4     0.50    (bread, milk)
5     0.50   (butter, milk)

==================================================
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski
0	(bread)	(butter)	0.75	0.75	0.5	0.666667	0.888889	1.0	-0.0625	0.75	-0.333333	0.5	-0.333333	0.666667
1	(butter)	(bread)	0.75	0.75	0.5	0.666667	0.888889	1.0	-0.0625	0.75	-0.333333	0.5	-0.333333	0.666667
2	(bread)	(milk)	0.75	0.75	0.5	0.666667	0.888889	1.0	-0.0625	0.75	-0.333333	0.5	-0.333333	0.666667
3	(milk)	(bread)	0.75	0.75	0.5	0.666667	0.888889	1.0	-0.0625	0.75	-0.333333	0.5	-0.333333	0.666667
4	(butter)	(milk)	0.75	0.75	0.5	0.666667	0.888889	1.0	-0.0625	0.75	-0.333333	0.5	-0.333333	0.666667
5	(milk)	(butter)	0.75	0.75	0.5	0.666667	0.888889	1.0	-0.0625	0.75	-0.333333	0.5	-0.333333	0.666667

Practical No - 9 : Demo of Open AI/TensorFlow tools

Aim:

- To explore OpenAI/TensorFlow tools and libraries.
- To demonstrate their capabilities through a mini-project.
- To present and discuss the findings.
- To identify potential applications of the tools.

Code:

```
import tensorflow as tf

import numpy as np

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

# Step 1: Prepare a dataset of labeled emails

emails = [

  "Buy cheap watches! Free shipping!",

  "Meeting for lunch today?",

  "Claim your prize! You've won $1,000,000!",

  "Important meeting at 3 PM."

]

labels = [1, 0, 1, 0]

# Step 2: Tokenize and pad the email text data

max_words = 1000

max_len = 50

tokenizer = Tokenizer(

  num_words=max_words,

  oov_token="<OOV>"

)

tokenizer.fit_on_texts(emails)

sequences = tokenizer.texts_to_sequences(emails)

X_padded = pad_sequences(
```

```
  sequences,
  maxlen=max_len,
  padding="post",
  truncating="post")
# Step 3: Define the neural network model
model = tf.keras.Sequential([
  tf.keras.layers.Embedding(
    input_dim=max_words,
    output_dim=16,
    input_length=max_len  ),
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dense(16, activation='relu'),
  tf.keras.layers.Dense(1, activation='sigmoid')
])
# Compile the model
model.compile(
  optimizer='adam',
  loss='binary_crossentropy',
  metrics=['accuracy'])
# Step 4: Define training data and labels as NumPy arrays
training_data = np.array(X_padded)
training_labels = np.array(labels)
# Step 5: Train the model
model.fit(
  training_data,
  training_labels,
  epochs=10
)
# Step 6: Test if 'Spam.txt' is spam or not
file_path = "SPAM.txt"
# Read the content of the 'Spam.txt' file
```

```
with open(file_path, "r", encoding="utf-8") as file:
  sample_email_text = file.read()
# Tokenize and pad the sample email text
sequences_sample = tokenizer.texts_to_sequences(
  [sample_email_text]
)
sample_email_padded = pad_sequences(
  sequences_sample,
  maxlen=max_len,
  padding="post",
  truncating="post"
)
# Use the trained model to make predictions
prediction = model.predict(sample_email_padded)
# Set a classification threshold
threshold = 0.5
# Classify the sample email based on the threshold
if prediction > threshold:
  print(f"Sample Email ('{file_path}'): SPAM")
else:
  print(f"Sample Email ('{file_path}'): NOT SPAM")
```

Output:

```
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.5000 - loss: 0.7027
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - accuracy: 0.5000 - loss: 0.6958
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - accuracy: 0.5000 - loss: 0.6920
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 62ms/step - accuracy: 0.5000 - loss: 0.6898
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step - accuracy: 0.5000 - loss: 0.6876
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 66ms/step - accuracy: 0.5000 - loss: 0.6855
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step - accuracy: 0.5000 - loss: 0.6833
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 90ms/step - accuracy: 0.5000 - loss: 0.6810
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step - accuracy: 0.5000 - loss: 0.6787
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - accuracy: 0.5000 - loss: 0.6763
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 99ms/step
Sample Email ('Spam.txt'): NOT SPAM
```